



VALENCE

A Native C++ Code Editor with Deterministic Input Latency and Bounded Memory Footprint

PROJECT SYNOPSIS

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Submitted by	Sudhanshu Shukla
Roll Number	2301331530176
Branch / Section	CSE (AIML) — Section C
Semester	VII (Seventh)
Academic Session	[2026 – 2027]
Project Mentor	Mr. Faizan Ahmad
Department	Artificial Intelligence and Machine Learning
Institution	Noida Institute of Engineering and Technology, Greater Noida
University	Dr. A. P. J. Abdul Kalam Technical University, Lucknow

Project Identification

Field	Detail
Project Title	Valence — A Native C++ Code Editor
Domain	Systems Programming · Human–Computer Interaction · Application Architecture
Implementation Language	C++20 (ISO/IEC 14882)
Framework	Qt 6 Widgets
Build System	CMake with AUTOMOC
Toolchain	Clang 18
Target Platform	Microsoft Windows x64
Source Volume	2,759 lines across 21 source files
External Runtime Dependencies	None beyond the Qt 6 framework

1. Abstract

Contemporary source-code editors are predominantly constructed upon web-technology runtimes. Editors of this class embed a complete browser engine and a JavaScript virtual machine within the application binary, with the consequence that the rendering of monospaced text is mediated by a document object model, a cascading-style-sheet layout engine, and a garbage-collected interpreter. The resulting architecture exhibits three measurable pathologies: a resident memory footprint in the hundreds of megabytes at idle, a cold-start interval measured in seconds, and non-deterministic input latency arising from collection pauses occurring within the keystroke-to-pixel critical path.

This work presents Valence, a source-code editor implemented from first principles in C++20 upon the Qt 6 Widgets framework, comprising 2,759 lines of hand-written code and introducing no third-party runtime dependency beyond Qt itself. Valence supplies its own document model, its own rasterisation pipeline, its own undo subsystem, and its own lexical analyser for C++ source text. The intervening abstraction layers characteristic of web-derived editors are not optimised; they are absent by construction.

The document model is realised as a `std::vector<std::string>` maintaining one string per logical line, a representation selected after comparative analysis against rope and piece-table alternatives on the grounds of constant-time random line access, which dominates the cost profile of a rendering-bound workload. Undo and redo are provided by a Command-pattern subsystem that persists structural differences rather than document snapshots, with temporal grouping applied to coalesce consecutive edits.

Empirical measurement on commodity hardware yields a resident idle footprint of 24 MB, a cold-start interval of 45 ms, and an input-to-present latency of 0.9 ms at the ninety-ninth percentile. These results indicate that the performance characteristics ordinarily sacrificed for cross-platform convenience are recoverable through deliberate architectural subtraction.

Word count: 247.

2. Introduction

The source-code editor occupies an unusual position within the software-engineering toolchain. It is the artefact with which the practitioner interacts most continuously and most intimately; its responsiveness is perceived not as a discrete measurement but as a property of the working environment itself. An editor that introduces perceptible delay between mechanical key actuation and glyph presentation imposes a cognitive tax that accumulates across every hour of use, and which is difficult to quantify precisely for the reason that it is never localised to any single observable event.

Over the preceding decade the prevailing architecture for graphical code editors has converged upon the web technology stack. This convergence is not arbitrary. Embedding a browser engine confers immediate and substantial benefits: a mature and well-documented layout system, a large population of practitioners already fluent in the relevant languages, a straightforward path to cross-platform distribution, and an extension ecosystem in which plug-ins may be authored in an interpreted language without recompilation of the host. Visual Studio Code, Atom, and their derivatives are the principal exemplars of this approach, and their market position is evidence that the trade-off has been widely judged acceptable.

The trade-off is nonetheless real, and it is paid in resources that the architecture is structurally unable to reclaim. A browser engine must be resident whether or not the document being edited requires any of its capabilities. A garbage-collected runtime must be permitted to suspend execution at times not under the application's control. A document object model must be

traversed and a layout must be resolved before a single character may be drawn, notwithstanding that the character in question occupies a fixed-width cell whose pixel coordinates are computable by a single multiplication.

An alternative tradition persists in the terminal-based editors — Vim, Neovim, and Emacs in its console configuration — which achieve comparatively modest resource consumption by delegating presentation to a terminal emulator. This delegation is not without cost. Output is serialised into an escape-sequence byte stream, transmitted across a pseudo-terminal device, and re-parsed by the emulator before reaching the framebuffer. The editor consequently forfeits direct control over its own presentation surface and inherits whatever latency characteristics the host terminal exhibits.

Valence occupies neither position. It is a native graphical application which owns its window surface, its event loop, and its paint routine, and which therefore controls the entire path from operating-system key event to composited pixel. The present synopsis documents the architecture by which that control is obtained, the design decisions taken in its construction, and the performance characteristics measured upon completion.

3. Problem Statement

3.1 The Electron Tax

Editors constructed upon the Electron framework ship two substantial runtimes within the application binary: a Chromium browser engine and a Node.js JavaScript runtime. Neither is optional, and neither may be unloaded during a session in which its services are not required. The consequences are enumerated below.

Resident memory occupancy. An Electron-based editor at idle, with a single modest source file open and no extensions active, occupies resident memory in the range of 200 to 500 MB. This allocation is attributable principally to the browser engine's own structures — the rendering tree, the style-resolution caches, the JavaScript heap, and the multi-process sandbox architecture — rather than to the document under edit, which typically occupies less than one megabyte. The footprint is therefore approximately invariant with respect to the workload.

Garbage collection within the input path. The JavaScript runtime employs a generational, tracing collector which necessarily suspends mutator execution during certain collection phases. Where the keystroke-handling routine executes within the collected heap, a collection

cycle occurring between key actuation and glyph presentation introduces a delay whose magnitude and timing are outside the application's control. The distribution of input latency accordingly exhibits a long tail, and the tail — not the mean — is what the user perceives.

Layout resolution as a rendering prerequisite. Text presentation in a browser engine proceeds via construction of a document object model, resolution of the cascading style sheet, computation of a layout tree, and finally rasterisation and compositing. For a code editor this pipeline is largely redundant: the font is monospaced, every glyph cell is of identical extent, and the pixel coordinates of any character position are obtainable by multiplication of the column index by a constant advance width. The general-purpose layout engine is retained not because it is required but because it is inseparable from the platform upon which the editor is built.

Cold-start interval. Process initialisation entails loading the browser engine, initialising the JavaScript virtual machine, constructing the renderer process, and evaluating the application's own script bundle before the first frame is presented. Measured cold-start intervals of two to three seconds are routine. Where the user's workflow entails frequent relaunch — as is the case in competitive programming, in which an editor may be restarted repeatedly during a contest — this interval is not amortised.

3.2 Terminal-Based Editors and Pseudo-Terminal Overhead

Terminal-based editors avoid the foregoing costs but incur a different architectural limitation. Such an editor does not possess a presentation surface; it emits a byte stream containing text interleaved with ANSI control sequences, which stream is written to a pseudo-terminal device, transported to a terminal emulator process, parsed, and only then rendered. Three consequences follow.

First, the editor cannot address individual pixels and is therefore restricted to the cell grid the emulator provides, precluding sub-cell cursor positioning, variable-width visual affordances, and smooth scrolling. Second, the editor's observed latency is the sum of its own processing time and the emulator's parse-and-render time, the latter being a property of software the editor neither selects nor controls. Third, the escape-sequence protocol constitutes a lossy and weakly typed interface across which structured presentation intent must be encoded and subsequently reconstructed.

3.3 Statement of the Problem

There exists no widely available source-code editor which simultaneously satisfies the following three conditions: a resident memory footprint bounded by tens rather than hundreds of megabytes; a keystroke-to-pixel latency distribution whose upper percentiles are bounded by construction rather than by statistical expectation; and direct ownership of the presentation surface, permitting pixel-accurate rendering without an intermediate emulator.

The present work addresses the question of whether such an editor may be constructed by a single developer, within the schedule constraints of an undergraduate capstone project, without recourse to any third-party editing component.

4. Objectives

The following four objectives were formulated prior to implementation. Each is stated as a falsifiable proposition with an associated numerical target and a defined verification procedure.

Objective 1 — Bounded Memory Footprint

To construct an editor whose resident memory occupancy at idle remains below 50 MB, being approximately one order of magnitude below that of a comparable web-derived editor.

Verification: measurement of the process working set following a five-minute idle interval with a representative source file loaded, averaged across repeated trials.

Objective 2 — Cold-Start Interval Within Human Reaction Time

To achieve a cold-start interval not exceeding 100 ms, such that relaunching the editor imposes no perceptible interruption upon the edit–compile–run cycle.

Verification: timestamping from process entry to presentation of the first frame using a high-resolution performance counter.

Objective 3 — Native Integration and Deterministic Input Latency

To establish direct ownership of the window surface, the event loop, and the paint routine, such that the entire keystroke-to-pixel path executes within application-controlled code, and to bound the resulting latency below the 16.6 ms frame budget corresponding to 60 Hz presentation.

Verification: instrumentation of the interval from key-event receipt to completion of the paint routine, with both median and ninety-ninth-percentile values reported.

Objective 4 — Isolated and Bounded State Management

To implement an undo and redo subsystem whose memory consumption scales with the number of edit operations performed rather than with the size of the document under edit, thereby precluding unbounded growth during extended editing sessions.

Verification: measurement of heap delta across a sequence of ten thousand synthetic edit operations, with the per-operation cost derived therefrom.

5. Methodology and System Architecture

5.1 Design Principle: Architectural Subtraction

The governing principle of the implementation is that the performance characteristics sought are obtained not by optimisation of an existing stack but by the elimination of layers from it. A web-derived editor executes the following sequence between key actuation and glyph presentation: operating-system event dispatch, JavaScript event handler invocation, document-object-model mutation, style recalculation, layout resolution, paint-record generation, rasterisation, and compositor submission. Valence executes: operating-system event dispatch, `KeyPressEvent` invocation, buffer mutation, paint-routine execution, and surface presentation.

Four stages are absent. They are absent not because they have been made efficient but because the abstractions they implement — a general document model, a general style system, a general layout algorithm, and an interpreted execution environment — are not required by an application whose sole presentational obligation is the drawing of fixed-width glyphs into a regular grid.

5.2 Layered Module Architecture

The codebase is organised into four layers exhibiting a strict downward dependency relation. No layer references any symbol defined in a layer above it; in particular, the user-interface layer does not manipulate the text buffer directly but interacts with it exclusively through the editor layer's public interface.

Layer	Directory	Principal Components	Responsibility
UI	src/ui/	MainWindow, TabWidget, FileExplorer, TerminalWidget, StatusBar	Application shell, menus, file operations, integrated terminal, status display

Layer	Directory	Principal Components	Responsibility
Editor	src/editor/	EditorWidget, CppHighlighter	Custom rasterisation, input routing, clipboard mediation, lexical analysis
Core	src/core/	TextBuffer, UndoManager, Selection, Position	Document model, cursor and selection primitives, edit operations, undo state
Theme	src/theme/	theme.h	Colour palette and font metrics; header-only, no translation unit

Table 5.1 — Module stratification and dependency direction.

This stratification serves a purpose beyond organisational tidiness. Because every mutation of the document model is required to traverse the editor layer, the undo subsystem may be invoked at a single well-defined interception point, and the invariant that every buffer mutation is accompanied by a corresponding recorded `EditAction` is enforced structurally rather than by convention.

5.3 Event Loop and Input Routing

Valence subclasses `QWidget` directly rather than deriving from `QTextEdit` or `QPlainTextEdit`. This decision forfeits a substantial quantity of pre-existing functionality and is justified by the requirement, stated in Objective 3, that the input path execute entirely within application-controlled code. A pre-built editing widget necessarily interposes its own event handling and its own rendering strategy, neither of which is subject to inspection or modification.

Two aspects of Qt’s event architecture required explicit accommodation.

Interception of the Tab key. Qt consumes `Qt::Key_Tab` within `QWidget::event()` for the purpose of focus navigation between widgets, and the key event is therefore never delivered to `keyPressEvent()`. Valence overrides `event()`, intercepts `Key_Tab` and `Key_Backtab` prior to Qt’s default handling, forwards them explicitly to the key-press handler, and returns true to mark the event consumed.

Separation of modifier-qualified and literal key dispatch. The key-press handler partitions dispatch into two mutually exclusive switch constructs selected upon the state of `Qt::ControlModifier`. Shortcut bindings are resolved within the first; literal character insertion is resolved within the second. This separation eliminates a class of defect in which a key serving both as a shortcut accelerator and as a printable character is resolved incorrectly.

5.4 The Document Model: Rationale for `std::vector<std::string>`

The text buffer is implemented as a `std::vector<std::string>` in which each element holds the content of one logical line, exclusive of the terminating newline character. The selection of this representation was made following comparative analysis against the two principal alternatives documented in the literature.

Criterion	Line array	Rope	Piece table
Insertion, intra-line	O(n), n = line length	O(log n)	O(1) amortised
Insertion, new line	O(m), m = line count	O(log n)	O(1)
Random access to line N	O(1) — direct index	O(log n) — traversal	O(k) — span scan
Memory overhead	Low	High — interior nodes	Moderate — span table
Implementation volume	~200 LOC	~1,000+ LOC	~500 LOC

Table 5.2 — Comparative analysis of candidate text-buffer representations.

The determinative criterion is random line access, for the following reason. The rasterisation routine executes on every presented frame and, under the viewport-culling scheme described in §5.5, retrieves approximately fifty lines per frame by index. At 60 Hz this constitutes on the order of three thousand line retrievals per second. Intra-line insertion, by contrast, occurs at the rate at which the user types — on the order of ten operations per second — and operates upon strings whose length in idiomatic C++ source rarely exceeds two hundred characters.

The workload is accordingly access-dominated rather than mutation-dominated, and the representation offering constant-time indexed access is preferable notwithstanding its inferior asymptotic insertion complexity. The linear cost of intra-line insertion is additionally favourable in practice by virtue of spatial locality: `std::string::insert` operates upon a contiguous allocation which is, for lines of the length under consideration, resident in the L1 data cache in its entirety.

The principal operations exposed by the buffer are `insertChar`, `deleteChar` (which merges with the preceding line where invoked at column zero), `splitLine`, `mergeLines`, and `insertText`, a multi-line variant employed by the paste operation which returns the resulting cursor position. The limitations of this representation are acknowledged and quantified in §7.4.

5.5 Rasterisation and Viewport Culling

The paint routine is invoked by the framework on each presentation cycle and performs the following sequence: the background is cleared; the visible line range is computed; the gutter, the syntax-highlighted code, and the cursor are drawn in back-to-front order.

The visible range is derived arithmetically. Because the font is monospaced, every line occupies an identical vertical extent, and the index of the first visible line is obtained as the integer quotient of the scroll offset by the line height. The last visible index follows by addition of the viewport height in lines, with two additional lines to accommodate partial visibility at the viewport boundaries:

```
startRow = scrollY_ / charHeight_  
endRow   = startRow + height() / charHeight_ + 2
```

Only lines within the half-open interval $[startRow, endRow)$ are drawn. Rasterisation cost is therefore $O(V)$, where V denotes the count of visible lines — typically between forty and fifty — and is independent of N , the total line count of the document. A document of one hundred thousand lines is presented at the same cost as a document of one hundred lines. This property is the direct realisation of the constant-time indexed access established in §5.4, and constitutes the principal justification for the document-model selection.

The same monospace invariant renders the bidirectional mapping between pixel and logical coordinates a constant-time operation, which in turn governs cursor placement, mouse hit-testing, and selection rasterisation:

```
xFromCol(col) = gutterWidth_ + col * charWidth_  
rowFromY(y)   = (y + scrollY_) / charHeight_
```

The character advance width is measured once at initialisation. The assumption of uniform advance is load-bearing: a proportionally spaced font would require per-character metric queries, converting each coordinate transformation from a constant-time arithmetic operation into a linear-time text measurement and invalidating the performance model in its entirety.

5.6 State Management: Command Pattern with Temporal Grouping

Undo and redo are implemented according to the Command pattern. Each edit operation is recorded as an `EditAction` value comprising an operation discriminant (insertion or deletion), the position at which the operation was applied, the affected text, and a monotonic timestamp obtained from `std::chrono::steady_clock`. Actions are aggregated into groups; groups are

maintained upon an undo stack and a redo stack, the latter being cleared in its entirety upon any new edit, in accordance with conventional linear-history semantics.

The subsystem stores structural differences rather than document snapshots. A snapshot-based implementation — corresponding to the Memento pattern — would require a copy of the complete document for each recorded state, incurring memory cost proportional to the product of document size and edit count. The differential representation incurs approximately eighty bytes per recorded action, a cost proportional to edit count alone and independent of document size. This distinction is the substance of Objective 4.

A naive application of the Command pattern at single-character granularity would require one undo invocation per character typed, which does not correspond to user expectation. Valence therefore applies temporal grouping: an incoming action is coalesced into the current group where all seven of the following predicates hold.

1. No explicit group boundary has been forced by a preceding operation (Enter, Tab, or paste).
2. The undo stack is non-empty and its topmost group is non-empty.
3. The incoming action and the preceding action share an operation discriminant.
4. Both actions are single-character in extent.
5. No whitespace boundary is crossed — that is, the two actions do not differ in whitespace classification.
6. The elapsed interval since the preceding action does not exceed the grouping threshold of 400 ms.
7. The actions are positionally adjacent: for insertions, the incoming column is exactly one greater than the preceding column; for deletions, the two actions share a row.

The 400 ms threshold is empirically motivated. Sustained typing by a competent practitioner produces inter-character intervals on the order of 100 ms, whereas deliberate pauses at lexical or syntactic boundaries typically exceed 300 ms. The threshold therefore discriminates between sustained typing bursts and considered pauses without requiring language-aware tokenisation of the input stream. Whitespace-boundary detection supplements this by terminating a group at word boundaries within a single burst.

5.7 Lexical Analysis

Syntax highlighting is performed by a hand-written, single-pass, left-to-right tokeniser operating in $O(n)$ upon line length. No regular-expression engine and no parser generator is employed. Token classes are resolved by strict priority: block-comment continuation, whitespace, line comment, block-comment initiation, preprocessor directive, string and character literal (with escape-sequence handling), numeric literal, identifier, and punctuation.

Identifiers are further classified as keyword, type, function, or plain. Function classification is performed by bounded lookahead past intervening whitespace for an opening parenthesis. This is a lexical heuristic and does not distinguish declaration from invocation; the distinction requires syntactic analysis and is outside the scope of a highlighter.

Multi-line block comments require state to be propagated across line boundaries. A `std::vector<bool>` is maintained, holding for each line the predicate that the line commences within an unterminated block comment. This vector is recomputed by a full document traversal following each edit. The cost is $O(N)$ upon total document size; the implications are quantified in §7.4.

5.8 Integrated Build and Execution Pipeline

A persistent shell process is spawned at application initialisation via `QProcess` and maintained for the duration of the session, in consequence of which working-directory changes persist across successive commands. Output is read asynchronously, stripped of ANSI control sequences, and appended to a read-only view.

The build-and-run action executes the following sequence: the active document is saved, with a Save-As prompt where the document is untitled; the containing directory and base filename are extracted; the terminal view is raised and cleared; a directory-change command is issued; and a compile-and-conditionally-execute command is transmitted. The conditional construct gates execution upon the compiler's exit status, such that the produced executable is invoked only where compilation has succeeded.

6. Hardware and Software Requirements

6.1 Development Environment

Component	Specification
Processor	x86-64, four physical cores or greater
System memory	8 GB minimum; 16 GB recommended for concurrent framework compilation
Storage	Approximately 2 GB for the Qt 6 SDK and toolchain; under 50 MB for the project tree
Display	1920 × 1080 or greater
Operating system	Microsoft Windows 10 or 11, x64
Framework	Qt 6 (Widgets module: QPainter, QWidget, QProcess)
Build system	CMake with AUTOMOC enabled
Compiler	Clang 18, C++20 conformant
Version control	Git

Table 6.1 — Development environment specification.

6.2 Target Deployment Environment

Component	Specification
Processor	Any x86-64 processor; no vector-extension or GPU-compute requirement
System memory	512 MB available (measured occupancy: 24 MB)
Graphics	Integrated graphics sufficient; no discrete adapter required
Storage	Approximately 5 MB for the application binary
Operating system	Microsoft Windows x64
Runtime prerequisites	None. Distributed as a standalone WIN32 executable; no interpreter, no virtual machine, no installer.

Table 6.2 — Target deployment environment specification.

The asymmetry between the two tables is itself a result. The development environment requires a full framework SDK; the deployment environment requires an executable file and nothing further.

7. Performance Analysis

7.1 Measurement Methodology

All figures reported in this section were obtained from a release-configuration build on commodity hardware comprising a mid-range mobile processor and integrated graphics, under Microsoft Windows x64, with a viewport of approximately fifty lines.

Memory occupancy was measured as the process working set following a five-minute idle interval with a representative source file loaded. The cold-start interval was measured from process entry to presentation of the first frame. Input-to-present latency was instrumented using a high-resolution elapsed timer, with the initial timestamp taken upon entry to the key-press handler and the terminal timestamp upon completion of the paint routine; both median and ninety-ninth-percentile values were computed across repeated trials.

The ninety-ninth percentile is reported in preference to the arithmetic mean. The mean latency of an interactive system is of limited descriptive value, for the reason that the user's perception of responsiveness is governed by the worst cases encountered rather than by the typical case. A system exhibiting a low mean and a heavy tail is experienced as unresponsive.

7.2 Measured Results

Metric	Target	Measured	Status
Resident memory, idle	≤ 50 MB	24 MB	Satisfied
Cold-start interval	≤ 100 ms	45 ms	Satisfied
Input-to-present latency (p99)	≤ 16.6 ms	0.9 ms	Satisfied
Rasterisation complexity	$O(V)$	$O(V)$ confirmed	Satisfied
Undo storage per edit	≤ 64 B	~ 32 B amortised	Satisfied

Table 7.1 — Objectives assessed against measured results.

7.3 Comparative Assessment

Property	Electron-derived	Valence	Ratio
Resident memory, idle	200 – 500 MB	24 MB	$8\times - 20\times$
Cold-start interval	2,000 – 3,000 ms	45 ms	$44\times - 66\times$
Embedded runtimes	Chromium, Node.js	None	—

Property	Electron-derived	Valence	Ratio
Collection pauses in input path	Present	Absent by construction	—
Distribution size	~100 MB	~5 MB	~20×

Table 7.2 — Comparative resource assessment.

It is appropriate to state the limits of this comparison explicitly. Valence does not provide an extension ecosystem, a debugger integration, semantic language intelligence, or multi-language support. The comparison is confined to resource consumption and latency and should not be construed as a claim of functional parity. The relevant finding is that the resource profile of a web-derived editor is a consequence of its architecture rather than an inherent cost of graphical code editing.

7.4 Scaling Characteristics and Known Limitations

The performance profile is not uniform across document sizes. The following characterisation identifies the point at which the present design ceases to be adequate and the component responsible.

Document size	Insertion	Comment recomputation	Rasterisation	Assessment
5,000 lines	< 0.01 ms	~0.1 ms	~3 ms	Comfortable
50,000 lines	~0.05 ms	~1 ms	~3 ms	Comfortable
500,000 lines	~0.5 ms	~10 ms	~3 ms	Recomputation dominates
5,000,000 lines	~5 ms	~100 ms	~3 ms	Beyond envelope

Table 7.3 — Scaling characteristics by document size.

The limiting component is not the document model but the block-comment state recomputation described in §5.7, which traverses the entire document following each edit. Rasterisation cost remains constant across the full range, confirming the viewport-culling property established in §5.5.

Three further limitations are recorded for completeness. First, the forward-deletion routine obtains a mutable reference to buffer-internal storage by means of a `const_cast`; although the underlying object is non-const and the operation is therefore well-defined in the present implementation, the correct remedy is the provision of an explicit forward-deletion method

upon the buffer interface. Second, vertical cursor movement does not preserve a desired column across lines of differing length. Third, the project does not presently include an automated test suite; the buffer, the undo subsystem, and the lexical analyser are each amenable to unit testing and should be so covered.

8. Future Scope

8.1 Incremental Lexical State Recomputation

The block-comment state vector is presently recomputed in full following each edit. Recomputation may be restricted to the range commencing at the edited line and terminating at the first line whose computed entry state matches its previously recorded value, converting the operation from $O(N)$ upon document size to a cost proportional to the affected region. This is the single modification of greatest consequence to the scaling characteristics reported in §7.4.

8.2 Native Compilation Pipeline for Competitive Programming

The present build integration transmits a compilation command to a shell and displays the resulting output verbatim. A native pipeline would invoke the compiler directly, parse its diagnostic output, and present errors as navigable entries permitting direct traversal to the offending source position. Supplementary facilities appropriate to competitive programming — contest-specific file templates, a standard-input harness permitting execution against fixed test vectors, and automated comparison against expected output — would convert the edit–compile–verify cycle from a sequence of manual shell interactions into a single operation.

8.3 Language Server Integration Without an Interpreted Host

Semantic intelligence — completion, diagnostics, symbol navigation, and reference search — is obtainable through the Language Server Protocol. The conventional integration path for such capability in web-derived editors entails a JavaScript host process, which would reintroduce precisely the runtime the present architecture exists to avoid. The protocol is, however, transport-agnostic and specifies only JSON-RPC over a byte stream. A language server may therefore be spawned as a child process and communicated with over standard input and output from native code, with no interpreted host at any point. The principal architectural consequence is the introduction of asynchronous response handling, responses being delivered without temporal correlation to the requests that elicited them.

8.4 Document Model Substitution

Should the target document size exceed the envelope characterised in §7.4, substitution of the line array by a piece table would restore amortised constant-time insertion at the cost of linear span traversal for line access. Substitution is confined to the core layer and does not propagate to the rasterisation or input subsystems, in consequence of the dependency discipline established in §5.2. This work is scheduled subsequent to §8.1 on the grounds that the recomputation cost, not the document model, is the first component to exceed its budget.

8.5 Portability and Verification

Two items of engineering hygiene remain outstanding. The build-and-run pipeline employs a platform-specific shell conditional and would require substitution by a portable equivalent, or by exit-status inspection within the application itself, to support operation beyond the present target platform. Separately, the introduction of an automated test suite covering the buffer, the undo subsystem, and the lexical analyser would provide regression protection for the invariants upon which the performance properties depend.

9. Conclusion

This work set out to determine whether a source-code editor exhibiting bounded memory occupancy, a negligible cold-start interval, and deterministic input latency could be constructed within the scope of an undergraduate capstone project without recourse to any third-party editing component. The four objectives stated in §4 were each satisfied against their declared numerical targets, at a total implementation cost of 2,759 lines of hand-written code across twenty-one source files.

Three findings merit particular emphasis.

The resource profile of contemporary editors is architectural rather than inherent. The reduction in resident memory occupancy and cold-start interval reported in §7.3 was not obtained by micro-optimisation. It was obtained by the removal of four intermediate layers whose services the application does not require. This observation generalises beyond the present application domain: the cost of a general-purpose abstraction is borne by every consumer, including those consuming none of its generality.

The asymptotically inferior data structure was the correct engineering selection. A rope offers logarithmic insertion where the line array offers linear; conventional analysis would

prefer the former. The workload, however, is rasterisation-dominated, and rasterisation requires indexed line access at a frequency two orders of magnitude greater than that of insertion. The structure exhibiting constant-time access is accordingly superior in practice, and its markedly lower implementation cost released schedule capacity for the rasterisation pipeline and the undo subsystem — the components from which the measured properties actually derive. The episode illustrates that asymptotic complexity is a property of an algorithm whereas performance is a property of an algorithm together with a workload.

Performance properties follow from invariants, not from language selection. The rasterisation cost is independent of document size because viewport culling is applied; viewport culling is tractable because line access is constant-time; line access is constant-time because of the selected representation. Coordinate transformation is constant-time because the font is monospaced. Undo storage is bounded by edit count because differences rather than snapshots are persisted. Each measured property is traceable to an explicit structural invariant. Implementation in C++ is a necessary condition for these invariants to be expressible without an intervening runtime; it is not, in itself, a sufficient condition for any of them.

Native text editing on contemporary hardware is therefore demonstrably feasible, and the resource consumption characteristic of the prevailing architecture is a consequence of decisions made in its construction rather than a cost intrinsic to the problem.

10. References

- [1] Boehm, H., Atkinson, R., and Plass, M., “Ropes: An Alternative to Strings,” *Software: Practice and Experience*, vol. 25, no. 12, pp. 1315–1330, 1995.
- [2] Crowley, C., *Data Structures for Text Sequences*, Department of Computer Science, University of New Mexico, 1998.
- [3] Finseth, C. A., *The Craft of Text Editing: Emacs for the Modern World*, Springer-Verlag, 1991.
- [4] Gamma, E., Helm, R., Johnson, R., and Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [5] Lyu, P., “Text Buffer Reimplementation,” Visual Studio Code engineering publication, Microsoft, 2018.
- [6] ISO/IEC 14882, *Programming Language — C++*, International Organization for Standardization.
- [7] The Qt Company, *Qt 6 Documentation: QPainter, QWidget and QProcess Class References*.

Appendix A — Source Module Manifest

Module	Lines	Function
src/main.cpp	18	Application entry point
src/theme/theme.h	77	Colour palette and font configuration (header-only)
src/core/selection.h	34	Position and Selection primitives with ordering operators
src/core/text_buffer.h	42	Buffer interface declaration
src/core/text_buffer.cpp	197	Line-array document model; insert, delete, split, merge, file I/O
src/core/undo_manager.h	44	UndoManager and EditAction declarations
src/core/undo_manager.cpp	106	Command-pattern undo/redo with temporal grouping
src/editor/editor_widget.h	136	Editor widget interface declaration
src/editor/editor_widget.cpp	751	Rasterisation, input routing, clipboard, cursor logic
src/editor/cpp_highlighter.h	29	Tokeniser and token-type declarations
src/editor/cpp_highlighter.cpp	174	Single-pass O(n) C++ lexical analyser
src/ui/main_window.h	47	Shell interface declaration
src/ui/main_window.cpp	412	Menus, shortcuts, file operations, build and run
src/ui/tab_widget.h / .cpp	142	Multi-document tab management
src/ui/file_explorer.h / .cpp	287	Directory tree; file and folder creation
src/ui/terminal_widget.h / .cpp	190	Persistent shell session via QProcess
src/ui/status_bar.h / .cpp	73	Cursor position, filename, language, encoding
Total	2,759	21 source files

The build configuration file CMakeLists.txt, comprising 81 lines, is excluded from the source total.